

Кафедра комп'ютерних
систем і мереж

ЛАБОРАТОРНА РОБОТА №13

з дисципліни **Теорія алгоритмів і структури даних**

Тема:

Способи задання графів

Виконала:

ст. гр. КІ-25-1

Лаврів А.

Перевірив:

Гарасимів Т.Г.

м. Івано-Франківськ

2026

Мета: Набуття практичних вмінь і навичок при представленні заданих графів різними способами та можливістю їх комп'ютерної реалізації.

Завдання:

Для неорієнтованих та орієнтованих графів зі своїх варіантів (варіант вибирається згідно номера по журналу, варіанти завдань наведено у додатку Б) виконати наступні завдання:

1. Написати програму, яка виконуватиме наступні завдання:

- за заданою матрицею суміжності побудувати матрицю інцидентності;
- за заданою матрицею інцидентності побудувати список ребер;
- за заданою матрицею суміжності побудувати список суміжності.
- за заданою матрицею інцидентності побудувати матрицю суміжності;
- за заданою матрицею суміжності побудувати список ребер;
- за заданою матрицею інцидентності побудувати список суміжності.

Код:

```
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include <algorithm>
```

```
using namespace std;
```

```
struct Edge {
    int from, to;
    bool loop;
    bool directed;
};
```

```
struct Graph {
    int n;
    vector<string> labels;
    vector<vector<int>> adjMatrix;
    bool isDirected;
```

```
Graph(int n, vector<string> labels, bool directed)
```

```

: n(n), labels(labels), isDirected(directed),
  adjMatrix(n, vector<int>(n, 0)) {}
};

```

```

void printSeparator(int width = 60) {
    cout << string(width, '-') << "\n";
}

```

```

void printTitle(const string& title) {
    cout << "\n";
    printSeparator();
    cout << " " << title << "\n";
    printSeparator();
}

```

```

void printAdjMatrix(const Graph& g) {
    printTitle("МАТРИЦЯ СУМІЖНОСТІ");
    cout << " ";
    for (int j = 0; j < g.n; j++)
        cout << setw(4) << g.labels[j];
    cout << "\n";
    for (int i = 0; i < g.n; i++) {
        cout << setw(4) << g.labels[i] << " ";
        for (int j = 0; j < g.n; j++)
            cout << setw(4) << g.adjMatrix[i][j];
        cout << "\n";
    }
    if (!g.isDirected)
        cout << "\nПримітка: матриця симетрична (неорієнтований граф)\n";
    else
        cout << "\nПримітка: матриця несиметрична (орієнтований граф)\n";
}

```

```

vector<Edge> buildEdgeList(const Graph& g) {

```

```
vector<Edge> edges;
```

```
if (!g.isDirected) {
```

```
    for (int i = 0; i < g.n; i++) {
```

```
        // Петли
```

```
        if (g.adjMatrix[i][i] > 0) {
```

```
            Edge e;
```

```
            e.from = i; e.to = i;
```

```
            e.loop = true; e.directed = false;
```

```
            edges.push_back(e);
```

```
        }
```

```
    for (int j = i + 1; j < g.n; j++) {
```

```
        int cnt = g.adjMatrix[i][j];
```

```
        for (int k = 0; k < cnt; k++) {
```

```
            Edge e;
```

```
            e.from = i; e.to = j;
```

```
            e.loop = false; e.directed = false;
```

```
            edges.push_back(e);
```

```
        }
```

```
    }
```

```
}
```

```
} else {
```

```
    for (int i = 0; i < g.n; i++) {
```

```
        if (g.adjMatrix[i][i] > 0) {
```

```
            for (int k = 0; k < g.adjMatrix[i][i]; k++) {
```

```
                Edge e;
```

```
                e.from = i; e.to = i;
```

```
                e.loop = true; e.directed = true;
```

```
                edges.push_back(e);
```

```
            }
```

```
        }
```

```
    for (int j = 0; j < g.n; j++) {
```

```
        if (i == j) continue;
```

```
        int cnt = g.adjMatrix[i][j];
```

```
        for (int k = 0; k < cnt; k++) {
```

```
            Edge e;
```

```

        e.from = i; e.to = j;
        e.loop = false; e.directed = true;
        edges.push_back(e);
    }
}
}
return edges;
}

```

```

void printIncidenceMatrix(const Graph& g, const vector<Edge>& edges) {
    printTitle("МАТРИЦЯ ІНЦИДЕНТНОСТІ");
    int m = (int)edges.size();

    cout << "    ";
    for (int j = 0; j < m; j++)
        cout << setw(4) << ("e" + to_string(j + 1));
    cout << "\n";

    for (int i = 0; i < g.n; i++) {
        cout << setw(4) << g.labels[i] << " ";
        for (int j = 0; j < m; j++) {
            const Edge& e = edges[j];
            int val = 0;
            if (!g.isDirected) {

                if (e.loop && e.from == i)
                    val = 2;
                else if (e.from == i || e.to == i)
                    val = 1;
            } else {

                if (e.loop && e.from == i)
                    val = 2;
                else if (e.from == i)
                    val = 1;
                else if (e.to == i)
                    val = -1;
            }
        }
    }
}

```

```

        cout << setw(4) << val;
    }
    cout << "\n";
}

if (!g.isDirected) {
    cout << "\nПримітка:\n";
    cout << " 1 = вершина інцидентна ребру\n";
    cout << " 2 = петля (псевдограф)\n";
} else {
    cout << "\nПримітка:\n";
    cout << " +1 = дуга виходить з вершини\n";
    cout << " -1 = дуга входить у вершину\n";
    cout << " 2 = петля (орієнтована)\n";
}
}

void printEdgeList(const Graph& g, const vector<Edge>& edges) {
    printTitle("СПИСОК РЕБЕР (ДУГ)");
    for (int j = 0; j < (int)edges.size(); j++) {
        const Edge& e = edges[j];
        cout << " e" << setw(2) << left << (j + 1) << ": ";
        if (!g.isDirected) {
            cout << "(" << g.labels[e.from] << ", " << g.labels[e.to] << ")";
        } else {
            cout << "(" << g.labels[e.from] << " -> " << g.labels[e.to] << ")";
        }
        if (e.loop)
            cout << " [петля]";
        cout << "\n";
    }
    cout << right;
    cout << "\nВсього ребер (дуг): " << edges.size() << "\n";
}

```

```

void printAdjList(const Graph& g) {

```

```

printTitle("СПИСОК СУМІЖНОСТІ");
for (int i = 0; i < g.n; i++) {
    cout << " " << g.labels[i] << ": ";
    bool empty = true;
    for (int j = 0; j < g.n; j++) {
        int cnt = g.adjMatrix[i][j];
        for (int k = 0; k < cnt; k++) {
            if (j == i) {
                cout << g.labels[j] << "(петля) ";
            } else {
                cout << g.labels[j] << " ";
            }
            empty = false;
        }
    }
    if (empty) cout << "(порожній — стік)";
    cout << "\n";
}
}

```

```

Graph incidenceToAdjMatrix(int n, vector<string> labels, bool directed,
                           const vector<vector<int>>& incMatrix, int m) {
    Graph g(n, labels, directed);
    for (int j = 0; j < m; j++) {
        if (!directed) {
            // Знаходимо вершини, інцидентні ребру
            vector<int> verts;
            int loopVert = -1;
            for (int i = 0; i < n; i++) {
                if (incMatrix[i][j] == 2) { loopVert = i; break; }
                if (incMatrix[i][j] == 1) verts.push_back(i);
            }
            if (loopVert >= 0) {
                g.adjMatrix[loopVert][loopVert]++;
            } else if (verts.size() == 2) {
                g.adjMatrix[verts[0]][verts[1]]++;
                g.adjMatrix[verts[1]][verts[0]]++;
            }
        }
    }
}

```

```

    } else {
        int from = -1, to = -1, loopVert = -1;
        for (int i = 0; i < n; i++) {
            if (incMatrix[i][j] == 2) { loopVert = i; break; }
            if (incMatrix[i][j] == 1) from = i;
            if (incMatrix[i][j] == -1) to = i;
        }
        if (loopVert >= 0)
            g.adjMatrix[loopVert][loopVert]++;
        else if (from >= 0 && to >= 0)
            g.adjMatrix[from][to]++;
    }
}
return g;
}

```

```

void incidenceToEdgeList(int n, const vector<string>& labels, bool directed,
                        const vector<vector<int>>& inc, int m) {
    printTitle("СПИСОК РЕБЕР (з матриці інцидентності)");
    for (int j = 0; j < m; j++) {
        cout << " e" << setw(2) << left << (j + 1) << ": ";
        if (!directed) {
            int loopVert = -1;
            vector<int> verts;
            for (int i = 0; i < n; i++) {
                if (inc[i][j] == 2) loopVert = i;
                if (inc[i][j] == 1) verts.push_back(i);
            }
            if (loopVert >= 0)
                cout << "(" << labels[loopVert] << ", " << labels[loopVert] << ") [петля]";
            else if (verts.size() == 2)
                cout << "(" << labels[verts[0]] << ", " << labels[verts[1]] << ")";
        } else {
            int from = -1, to = -1, loopVert = -1;
            for (int i = 0; i < n; i++) {
                if (inc[i][j] == 2) loopVert = i;
                if (inc[i][j] == 1) from = i;
                if (inc[i][j] == -1) to = i;
            }

```



```

    }
    if (loopVert >= 0)
        cout << "(" << labels[loopVert] << " -> " << labels[loopVert] << ") [петля]";
    else if (from >= 0 && to >= 0)
        cout << "(" << labels[from] << " -> " << labels[to] << ")";
    }
    cout << "\n";
}
cout << right;
}

```

```

void adjMatrixToAdjList(const Graph& g) {

```

```

    printTitle("СПИСОК СУМІЖНОСТІ (з матриці суміжності)");
    for (int i = 0; i < g.n; i++) {
        cout << " " << g.labels[i] << ": ";
        bool empty = true;
        for (int j = 0; j < g.n; j++) {
            int cnt = g.adjMatrix[i][j];
            for (int k = 0; k < cnt; k++) {
                cout << g.labels[j];
                if (j == i) cout << "(петля)";
                cout << " ";
                empty = false;
            }
        }
        if (empty) cout << "∅";
        cout << "\n";
    }
}

```

```

Graph makeUndirectedGraph11() {
    int n = 6;
    vector<string> labels = {"a","b","c","d","e","f"};
    Graph g(n, labels, false);
    //   a b c d e f
    g.adjMatrix[0] = {0, 2, 1, 0, 0, 0}; // a

```

```

g.adjMatrix[1] = {2, 1, 0, 1, 0, 0}; // b (1 = петля b-b)
g.adjMatrix[2] = {1, 0, 0, 2, 1, 0}; // c
g.adjMatrix[3] = {0, 1, 2, 0, 0, 1}; // d
g.adjMatrix[4] = {0, 0, 1, 0, 0, 1}; // e
g.adjMatrix[5] = {0, 0, 0, 1, 1, 0}; // f
return g;
}

```

```

Graph makeDirectedGraph11() {
    int n = 6;
    vector<string> labels = {"a","b","c","d","e","f"};
    Graph g(n, labels, true);

    g.adjMatrix[0] = {0, 2, 1, 0, 0, 0}; // a
    g.adjMatrix[1] = {0, 0, 1, 1, 0, 0}; // b
    g.adjMatrix[2] = {0, 0, 0, 1, 0, 1}; // c
    g.adjMatrix[3] = {0, 0, 0, 0, 0, 1}; // d
    g.adjMatrix[4] = {0, 0, 0, 1, 0, 1}; // e
    g.adjMatrix[5] = {0, 0, 0, 0, 0, 0}; // f
    return g;
}

```

```

void demoIncidenceToAdj_Undirected() {

    int n = 6, m = 10;
    vector<string> labels = {"a","b","c","d","e","f"};

    vector<vector<int>> inc = {

        { 1, 1, 1, 0, 0, 0, 0, 0, 0, 0 },
        { 1, 1, 0, 1, 2, 0, 0, 0, 0, 0 },
        { 0, 0, 1, 0, 0, 1, 1, 1, 0, 0 },
        { 0, 0, 0, 1, 0, 1, 1, 0, 1, 0 },
        { 0, 0, 0, 0, 0, 0, 0, 1, 0, 1 },
        { 0, 0, 0, 0, 0, 0, 0, 0, 1, 1 },
    };
}

```

```
printTitle("ДЕМО: матриця інцидентності → матриця суміжності  
(неорієнтований)");
```

```
cout << "Задана матриця інцидентності:\n";  
cout << "    ";  
for (int j = 0; j < m; j++) cout << setw(4) << ("e"+to_string(j+1));  
cout << "\n";  
for (int i = 0; i < n; i++) {  
    cout << setw(5) << labels[i] << " ";  
    for (int j = 0; j < m; j++) cout << setw(4) << inc[i][j];  
    cout << "\n";  
}
```

```
Graph recovered = incidenceToAdjMatrix(n, labels, false, inc, m);  
cout << "\nВідновлена матриця суміжності:\n";  
printAdjMatrix(recovered);
```

```
incidenceToEdgeList(n, labels, false, inc, m);  
adjMatrixToAdjList(recovered);  
}
```

```
void demoIncidenceToAdj_Directed() {  
    int n = 6, m = 10;  
    vector<string> labels = {"a","b","c","d","e","f"};  
    vector<vector<int>> inc = {  
  
        { 1, 1, 1, 0, 0, 0, 0, 0, 0, 0 }, // a  
        {-1, -1, 0, 1, 1, 0, 0, 0, 0, 0 }, // b  
        { 0, 0, -1, -1, 0, 1, 1, 0, 0, 0 }, // c  
        { 0, 0, 0, 0, -1, -1, 0, 1, -1, 0 }, // d  
        { 0, 0, 0, 0, 0, 0, 0, 0, 1, 1 }, // e  
        { 0, 0, 0, 0, 0, 0, -1, -1, 0, -1 }, // f  
    };  
}
```

```
printTitle("ДЕМО: матриця інцидентності → матриця суміжності (орієнтований)");  
cout << "Задана матриця інцидентності:\n";  
cout << "    ";  
for (int j = 0; j < m; j++) cout << setw(4) << ("e"+to_string(j+1));  
cout << "\n";  
for (int i = 0; i < n; i++) {  
    cout << setw(5) << labels[i] << " ";
```

```
    for (int j = 0; j < m; j++) cout << setw(4) << inc[i][j];  
    cout << "\n";  
}
```

```
Graph recovered = incidenceToAdjMatrix(n, labels, true, inc, m);  
cout << "\nВідновлена матриця суміжності:\n";  
printAdjMatrix(recovered);
```

```
incidenceToEdgeList(n, labels, true, inc, m);  
adjMatrixToAdjList(recovered);  
}
```

```
int main() {
```

```
    cout << " НЕОРІЄНТОВАНИЙ ГРАФ \n";
```

```
    cout << "Граф: a-b(×2), a-c, b-b(петля), b-d, c-d(×2), c-e, d-f, e-f\n";
```

```
    Graph ug = makeUndirectedGraph11();  
    printAdjMatrix(ug);
```

```
    vector<Edge> uEdges = buildEdgeList(ug);  
    printIncidenceMatrix(ug, uEdges);  
    printEdgeList(ug, uEdges);  
    printAdjList(ug);
```

```
    cout << "\n\n--- Перетворення з матриці інцидентності ---\n";  
    demoIncidenceToAdj_Undirected();
```

```
    cout << " ОРІЄНТОВАНИЙ ГРАФ \n";
```

```
    cout << "Граф: a→b(×2), a→c, b→c, b→d, c→d, c→f, d→f, e→d, e→f\n";
```

```

Graph dg = makeDirectedGraph11();
printAdjMatrix(dg);

vector<Edge> dEdges = buildEdgeList(dg);
printIncidenceMatrix(dg, dEdges);
printEdgeList(dg, dEdges);
printAdjList(dg);

cout << "\n\n--- Перетворення з матриці інцидентності ---\n";
demoIncidenceToAdj_Directed();

cout << " Усі перетворення виконано успішно.\n";

return 0;
}

```

```

НЕОРІЄНТОВАНИЙ ГРАФ
Граф: a-b(x2), a-c, b-b(петля), b-d, c-d(x2), c-e, d-f, e-f
-----
МАТРИЦЯ СУМІЖНОСТІ
-----
      a  b  c  d  e  f
a  0  2  1  0  0  0
b  2  1  0  1  0  0
c  1  0  0  2  1  0
d  0  1  2  0  0  1
e  0  0  1  0  0  1
f  0  0  0  1  1  0

Примітка: матриця симетрична (неорієнтований граф)
-----
МАТРИЦЯ ІНЦИДЕНТНОСТІ
-----
      e1  e2  e3  e4  e5  e6  e7  e8  e9  e10
a  1  1  1  0  0  0  0  0  0  0
b  1  1  0  2  1  0  0  0  0  0
c  0  0  1  0  0  1  1  1  0  0
d  0  0  0  0  1  1  1  0  1  0
e  0  0  0  0  0  0  0  1  0  1
f  0  0  0  0  0  0  0  0  1  1

Примітка:
1 = вершина інцидентна ребру
2 = петля (псевдограф)

```

Рис. 1 - Демонстрація роботи коду

```

-----
СПИСОК РЕБЕР (ДУГ)
-----
e1 : (a, b)
e2 : (a, b)
e3 : (a, c)
e4 : (b, b) [петля]
e5 : (b, d)
e6 : (c, d)
e7 : (c, d)
e8 : (c, e)
e9 : (d, f)
e10: (e, f)

Всього ребер (дуг): 10

-----
СПИСОК СУМІЖНОСТІ
-----
a: b b c
b: a a b(петля) d
c: a d d e
d: b c c f
e: c f
f: d e

--- Перетворення з матриці інцидентності ---

```

Рис.2 - Демонстрація роботи коду

```

-----
ДЕМО: матриця інцидентності → матриця суміжності (неорієнтований граф)
-----
Задана матриця інцидентності:
      e1 e2 e3 e4 e5 e6 e7 e8 e9 e10
a      1  1  1  0  0  0  0  0  0  0
b      1  1  0  1  2  0  0  0  0  0
c      0  0  1  0  0  1  1  1  0  0
d      0  0  0  1  0  1  1  0  1  0
e      0  0  0  0  0  0  0  0  1  0
f      0  0  0  0  0  0  0  0  0  1

Відновлена матриця суміжності:

-----
МАТРИЦЯ СУМІЖНОСТІ
-----
      a  b  c  d  e  f
a      0  2  1  0  0  0
b      2  1  0  1  0  0
c      1  0  0  2  1  0
d      0  1  2  0  0  1
e      0  0  1  0  0  1
f      0  0  0  1  1  0

Примітка: матриця симетрична (неорієнтований граф)

```

Рис. 3 - Демонстрація роботи коду

```

-----
СПИСОК РЕБЕР (з матриці інцидентності)
-----
e1 : (a, b)
e2 : (a, b)
e3 : (a, c)
e4 : (b, d)
e5 : (b, b) [петля]
e6 : (c, d)
e7 : (c, d)
e8 : (c, e)
e9 : (d, f)
e10: (e, f)

-----
СПИСОК СУМІЖНОСТІ (з матриці суміжності)
-----
a: b b c
b: a a b(петля) d
c: a d d e
d: b c c f
e: c f
f: d e
ОРІЄНТОВАНИЙ ГРАФ
Граф: a→b(x2), a→c, b→c, b→d, c→d, c→f, d→f, e→d, e→f

-----
МАТРИЦЯ СУМІЖНОСТІ
-----
      a  b  c  d  e  f
a    0  2  1  0  0  0
b    0  0  1  1  0  0
c    0  0  0  1  0  1
d    0  0  0  0  0  1
e    0  0  0  1  0  1
f    0  0  0  0  0  0

Примітка: матриця несиметрична (орієнтований граф)

```

Рис. 4 - Демонстрація роботи коду

```

-----
МАТРИЦЯ ІНЦИДЕНТНОСТІ
-----
      e1 e2 e3 e4 e5 e6 e7 e8 e9 e10
a     1  1  1  0  0  0  0  0  0  0
b    -1 -1  0  1  1  0  0  0  0  0
c     0  0 -1 -1  0  1  1  0  0  0
d     0  0  0  0 -1 -1  0  1 -1  0
e     0  0  0  0  0  0  0  0  1  1
f     0  0  0  0  0  0 -1 -1  0 -1

Примітка:
+1 = дуга виходить з вершини
-1 = дуга входить у вершину
 2 = петля (орієнтована)

-----
СПИСОК РЕБЕР (ДУГ)
-----
e1 : (a -> b)
e2 : (a -> b)
e3 : (a -> c)
e4 : (b -> c)
e5 : (b -> d)
e6 : (c -> d)
e7 : (c -> f)
e8 : (d -> f)
e9 : (e -> d)
e10: (e -> f)

Всього ребер (дуг): 10

```

Рис. 5 - Демонстрація роботи коду

```

-----
СПИСОК СУМІЖНОСТІ
-----

a: b b c
b: c d
c: d f
d: f
e: d f
f: (порожній – стік)

--- Перетворення з матриці інцидентності ---

-----
ДЕМО: матриця інцидентності → матриця суміжності (орієнтований)
-----

Задана матриця інцидентності:
      e1 e2 e3 e4 e5 e6 e7 e8 e9 e10
a      1  1  1  0  0  0  0  0  0  0
b     -1 -1  0  1  1  0  0  0  0  0
c      0  0 -1 -1  0  1  1  0  0  0
d      0  0  0  0 -1 -1  0  1 -1  0
e      0  0  0  0  0  0  0  0  1  1
f      0  0  0  0  0  0 -1 -1  0 -1

Відновлена матриця суміжності:

```

Рис. 6 - Демонстрація роботи коду

```

-----
МАТРИЦЯ СУМІЖНОСТІ
-----

      a  b  c  d  e  f
a      0  2  1  0  0  0
b      0  0  1  1  0  0
c      0  0  0  1  0  1
d      0  0  0  0  0  1
e      0  0  0  1  0  1
f      0  0  0  0  0  0

Примітка: матриця несиметрична (орієнтований граф)

-----
СПИСОК РЕБЕР (з матриці інцидентності)
-----

e1 : (a -> b)
e2 : (a -> b)
e3 : (a -> c)
e4 : (b -> c)
e5 : (b -> d)
e6 : (c -> d)
e7 : (c -> f)
e8 : (d -> f)
e9 : (e -> d)
e10: (e -> f)

-----
СПИСОК СУМІЖНОСТІ (з матриці суміжності)
-----

a: b b c
b: c d
c: d f
d: f
e: d f
f: 
Усі перетворення виконано успішно.

```

Рис. 7 - Демонстрація роботи коду

Висновок: У ході виконання лабораторної роботи було досліджено різні способи представлення графів: матрицю суміжності, матрицю інцидентності, список суміжності та список ребер. Було реалізовано перетворення між різними формами представлення графів та отримано практичні навички роботи з графовими структурами даних мовою C++.